

Agenda

- ① what is a function
- ② Function call and Function definition
- ③ Predetined and Userdefined function
- ④ Flow of program with multiple functions
- ⑤ Benefits of functions
- ⑥ Ways to define a function
- ⑦ Header files and Library files

What is a function?

- Function is a basic building block of a C program.
- Function is a block of code, which has some name for identification.
- A C program can have any number of functions

```
functionName( )  
{  
    // some code  
}
```

} Function
Definition

- Function names must be unique in a program
- Function definition vs function call

```
f1()
```

```
{
```

```
    printf("Hello");
```

```
    printf("I am a function");
```

```
}
```

```
{  
==
```

```
f1();
```

```
==
```

```
}
```

- Function is a way to implement modularization.
- Modularization is splitting up of a bigger task into several smaller sub-tasks to reduce the complexity of a problem.
- You can compile a C file without having `main()` function but cannot run.

Functions are of two types

1. Predefined Functions `printf()` `scanf()`
2. User defined Functions

`main()`

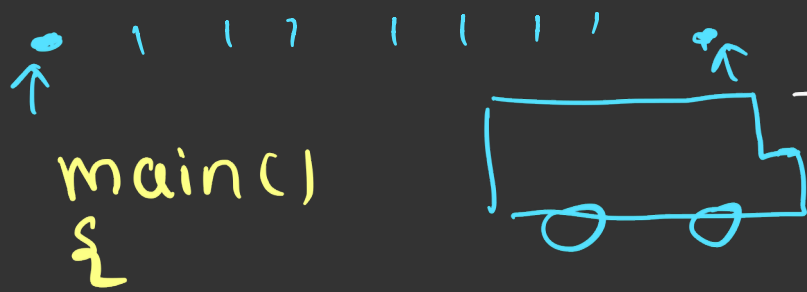
`{`

`—`
`—`
`—`
`—`
`—`

`}`

<code>switch()</code>	<code>x</code>
<code>while()</code>	<code>x</code>
<code>if()</code>	<code>x</code>
<code>sizeof()</code>	<code>x</code>
<code>return()</code>	<code>x</code>
<code>clrscr()</code>	<code>✓</code>

Flow of a program



```
main()
{
    a();
    b();
    a();
}
a()
{
    printf("Hello");
}
b()
{
    printf("Bye");
    a();
}
```

program's memory

Hello Bye Hello Hello

```
printf()
{
    //
}
```

Benefits of Functions

1. Easy to Read
2. Reduce Complexity
3. Easy to modify
4. Easy to debug
5. Code reusability
6. Avoids rewriting
7. Better memory utilization

```
a()
{
    int x;

}

b()
{
    int y;

}
```

Ways to define a function

1. Takes Nothing, Returns Nothing
2. Take something, Returns Nothing
3. Takes Nothing, Return Something
4. Take something, Return Something

TNRN

```
#include <stdio.h>
```

```
void add();  
int main()  
{  
    add();  
    return 0;  
}
```

← Function Declaration
or
Function Prototype

← Function call

```
void add() TN
```

```
{  
    int a, b, c;  
    printf("Enter two numbers");  
    scanf("%d %d", &a, &b);  
    c = a + b;  
    printf("Sum is %d", c);  
}
```

no return keyword = RN

Function
Definition

void add(int, int); TSRN

int main()

{

int x, y;

printf("Enter two numbers");

scanf("%d %d", &x, &y);

add(x, y); ← Actual Argument

return 0;

}

void add(int a, int b) ← Formal Arguments

{

int c;

c = a + b;

printf("sum is %d", c);

}

T N R S

```
int add();  
int main()  
{  
    int s;  
    s = add();  
    printf("Sum is %d", s);  
    return(0);  
}  
int add()  
{  
    int a, b, c;  
    printf("Enter two numbers");  
    scanf("%d %d", &a, &b);  
    c = a + b;  
    return c;  
}
```

TSRS

```
int add(int, int);
```

```
int main()
```

```
{
```

```
    int s, x, y;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d", &x, &y);
```

```
    s = add(x, y);
```

```
    printf("Sum is %d", s);
```

```
    return 0;
```

```
}
```

```
int add(int a, int b)
```

```
{
```

```
    int c;
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

Function call by
passing values.
(call by value)

Ask Yourself

1. Function call vs Function Definition vs Function Declaration
2. Function saves memory
3. `main()` is a userdefined function.
4. Header file vs Library file
5. When to write `void`?
6. When to write `return`?
7. Return type of `main()`
8. call by value
9. Function Prototype
10. Can we call `main()` function?

